Milestone 1 Report: International Hotel Booking Analytics

Mennatullah Essam Omar 55-1132
Sylvia Faris Wardkhan 55-24910
Mohamed Walid Mohamed Sherif 55-0810
Shahd Mohamed Fawzy 55-0551

October 24, 2025

1 Abstract

It is a project based on analyzing the international hotel booking data where geographical areas are predicted according to the experience of the users based on their reviews, ratings, and demographics. Three machine learning models were used and compared: Logistic Regression, Random Forest, and Neural Network. The highest test accuracy of 37% that the Random Forest had in 11-class regional classification was obtained on the test. The review scores showed in the feature importance analysis as the key predictors, especially the overall rating and value-for-money perceptions.

2 Introduction

In this project, the data on the international hotel bookings is analyzed to forecast the geographical area (country group) in which a hotel is situated (based on the user reviews, ratings, and other demographical factors). The dataset is made up of three broad sources of data namely hotels, users and reviews ,which were combined to form a complex analytical dataset.

2.1 Project Objective

Build predictive models to classify group hotels into regional groups (e.g., North America, Western Europe, East Asia) using features.

3 Methodology

3.1 Data Preparation and Cleaning

Data Merging: Combined three datasets (hotels, users, reviews) using `hotel_id` and `user_id` keys, resulting in a merged dataset for comprehensive analysis.

3.2 Data Preparation and Cleaning

Data Merging: Combined three datasets (`hotels`, `users`, `reviews`) using the shared keys `hotel_id` and `user_id`, resulting in a unified dataset for comprehensive analysis.

Data Quality Checks:

To ensure the reliability and integrity of the dataset, a series of data inspection and validation procedures were conducted as follows:

1. **Inspecting the Dataset Structure:**
   - The dataset shape and metadata were inspected using:

```
print("Dataset shape:", merged_df.shape)
print(merged_df.info())
```

The dataset contained **50,000 rows and 29 columns**, with both numerical and categorical attributes. The output showed no missing values, with 15 float columns, 4 integer columns, and 10 object columns. The total memory usage was approximately **11.1 MB**. as shown

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 29 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   review_id             50000 non-null  int64
 1   user_id               50000 non-null  int64
 2   hotel_id              50000 non-null  int64
 3   review_date           50000 non-null  object
 4   score_overall         50000 non-null  float64
 5   score_cleanliness     50000 non-null  float64
 6   score_comfort         50000 non-null  float64
 7   score_facilities      50000 non-null  float64
 8   score_location        50000 non-null  float64
 9   score_staff           50000 non-null  float64
 10  score_value_for_money 50000 non-null  float64
 11  review_text           50000 non-null  object
 12  hotel_name            50000 non-null  object
 13  city                  50000 non-null  object
 14  country_x             50000 non-null  object
 15  star_rating           50000 non-null  int64
 16  lat                   50000 non-null  float64
 17  lon                   50000 non-null  float64
 18  cleanliness_base      50000 non-null  float64
 19  comfort_base          50000 non-null  float64
 20  facilities_base       50000 non-null  float64
 21  location_base         50000 non-null  float64
 22  staff_base            50000 non-null  float64
 23  value_for_money_base  50000 non-null  float64
 24  user_gender           50000 non-null  object
 25  country_y             50000 non-null  object
 26  age_group             50000 non-null  object
 27  traveller_type        50000 non-null  object
 28  join_date             50000 non-null  object
dtypes: float64(15), int64(4), object(10)
memory usage: 11.1+ MB
None
```

2. **Descriptive Statistics:**
   - Summary statistics for numerical and categorical features were generated using:

```
print(merged_df.describe())
print(merged_df.describe(include=['object']))
```

These commands produced statistical summaries (mean, std, min, max) and frequency counts for categorical variables

	review_id	user_id	hotel_id	score_overall	\
count	50000.000000	50000.000000	50000.000000	50000.000000	
mean	25000.500000	1005.567540	13.051100	8.943460	
std	14433.901067	576.711855	7.203808	0.180878	
min	1.000000	1.000000	1.000000	8.200000	
25%	12500.750000	507.000000	7.000000	8.800000	
50%	25000.500000	1010.000000	13.000000	8.900000	
75%	37500.250000	1504.000000	19.000000	9.100000	
max	50000.000000	2000.000000	25.000000	9.600000	

	score_cleanliness	score_comfort	score_facilities	score_location	\
count	50000.000000	50000.000000	50000.000000	50000.000000	
mean	9.052558	9.024404	8.743062	9.176410	
std	0.504296	0.423927	0.498320	0.421086	
min	7.700000	7.900000	7.600000	7.900000	
25%	8.700000	8.700000	8.400000	8.900000	
50%	9.100000	9.000000	8.700000	9.200000	
75%	9.400000	9.300000	9.100000	9.500000	
max	10.000000	10.000000	10.000000	10.000000	

	score_staff	score_value_for_money	star_rating	lat	\
count	50000.000000	50000.000000	50000.0	50000.000000	
mean	8.972076	8.434540	5.0	20.882112	
std	0.397775	0.529321	0.0	30.437484	
min	7.900000	6.800000	5.0	-41.286500	
25%	8.700000	8.100000	5.0	6.524400	
50%	9.000000	8.500000	5.0	31.230400	
75%	9.200000	8.800000	5.0	41.902800	
max	10.000000	9.700000	5.0	55.755800	

	lon	cleanliness_base	comfort_base	facilities_base	\
count	50000.000000	50000.000000	50000.000000	50000.000000	
mean	34.468497	9.091592	9.063918	8.917196	
std	73.576721	0.226334	0.234129	0.286064	
min	-99.133200	8.700000	8.600000	8.500000	
25%	2.173400	8.900000	8.900000	8.700000	
50%	18.424100	9.100000	9.100000	8.900000	
75%	100.501800	9.300000	9.200000	9.100000	
max	174.776200	9.600000	9.500000	9.600000	

	location_base	staff_base	value_for_money_base
count	50000.000000	50000.000000	50000.000000
mean	9.270406	9.015730	8.513232
std	0.347649	0.236695	0.258259
min	8.500000	8.600000	7.900000
25%	9.000000	8.800000	8.400000
50%	9.300000	9.000000	8.500000
75%	9.600000	9.200000	8.700000
max	9.800000	9.500000	8.900000

	review_date	review_text	\
count	50000	50000	
unique	2192	50000	
top	2025-07-17	Practice reduce young our because machine. Rec...	
freq	50	1	

	hotel_name	city	country_x	user_gender	country_y	age_group	\
count	50000	50000	50000	50000	50000	50000	
unique	25	25	25	3	25	5	
top	Nile Grandeur	Cairo	Egypt	Male	United States	25-34	
freq	2104	2104	2104	23630	6971	16322	

	traveller_type	join_date
count	50000	50000
unique	4	1200
top	Couple	2023-01-17
freq	17240	174

3. **Null Value Analysis:**

- To confirm completeness, the dataset was examined for missing data:

```
import missingno as msno
```

```
msno.matrix(merged_df)
print(merged_df.isnull().sum())
```

The visualization and output confirmed that the dataset contained **no null values**.

```
review_id               0
user_id                 0
hotel_id                0
review_date             0
score_overall           0
score_cleanliness       0
score_comfort           0
score_facilities        0
score_location          0
score_staff             0
score_value_for_money   0
review_text             0
hotel_name              0
city                    0
country_x               0
star_rating             0
lat                     0
lon                     0
cleanliness_base        0
comfort_base            0
facilities_base         0
location_base           0
staff_base              0
value_for_money_base    0
user_gender             0
country_y               0
age_group               0
traveller_type          0
join_date               0
dtype: int64
```

+ Code + Markdown

NO null values

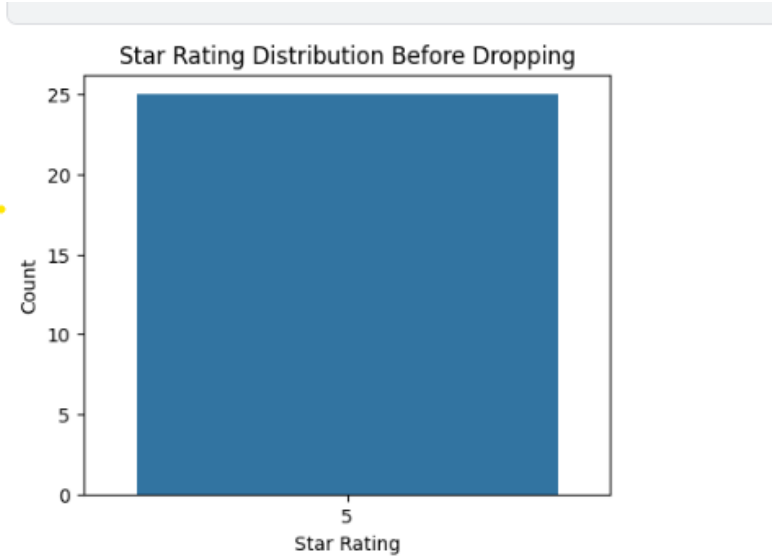
4. **Duplicate and Constant Column Checks:**
   - Checked for duplicate column names:

```
print(merged_df.columns[merged_df.columns.duplicated()])
```

   - Checked for constant columns (same value in all rows):

```
constant_cols = [col for col in merged_df.columns if merged_df[
    col].nunique() == 1]
print("Constant columns:", constant_cols)
```

The `star_rating` column was constant (value = 5), and was later removed.

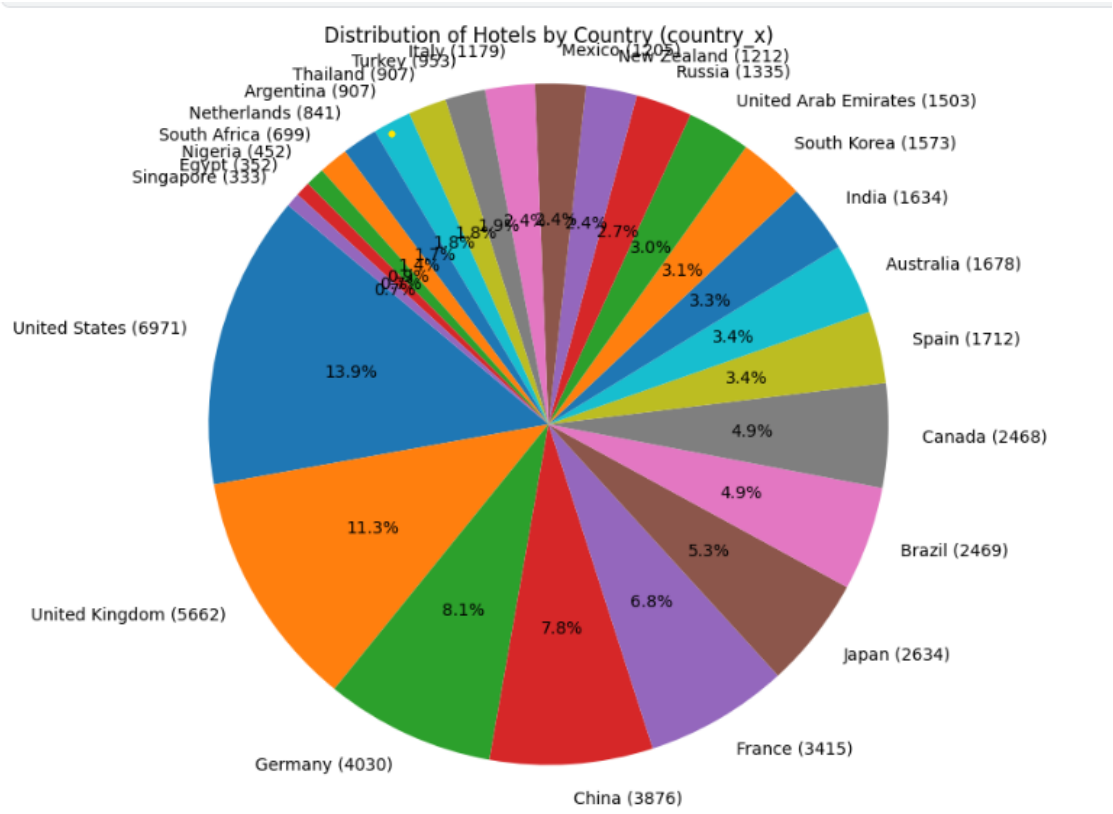


5. **Data Consistency Validation:**
   - Verified category consistency:
     - `traveller_type` → ['Business', 'Couple', 'Family', 'Solo']
     - `user_gender` → ['Male', 'Female', 'Other']
     - `country_x` contained valid country names
     - All score columns were within range 1–10

     All checks passed successfully.
6. **Duplicate Rows:** Verified there were no duplicate entries:

```
merged_df.duplicated().sum()
```

7. **Country Distribution Analysis:** Visualized the distribution of hotels by country to assess balance and representation.



8. **Removing Irrelevant Columns:** Dropped non-predictive or redundant features to prevent target leakage:
   - Identifiers: `review_id`, `hotel_id`, `user_id`
   - Temporal: `review_date`, `join_date`
   - Text: `review_text`
   - Geographic: `lat`, `lon`
   - Constant: `star_rating`
9. **Encoding Categorical Variables:**
   - Label encoded `user_gender` (Male/Female/Other $\rightarrow$ 0/1/2)
   - One-hot encoded `age_group` and `traveller_type`

   The structure after encoding is shown below.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 27 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   user_id                  50000 non-null  int64
 1   score_overall            50000 non-null  float64
 2   score_cleanliness        50000 non-null  float64
 3   score_comfort            50000 non-null  float64
 4   score_facilities         50000 non-null  float64
 5   score_location           50000 non-null  float64
 6   score_staff              50000 non-null  float64
 7   score_value_for_money    50000 non-null  float64
 8   hotel_name               50000 non-null  object
 9   city                     50000 non-null  object
 10  country_x                50000 non-null  object
 11  cleanliness_base         50000 non-null  float64
 12  comfort_base             50000 non-null  float64
 13  facilities_base          50000 non-null  float64
 14  location_base            50000 non-null  float64
 15  staff_base               50000 non-null  float64
 16  value_for_money_base     50000 non-null  float64
 17  user_gender_encoded      50000 non-null  int64
 18  age_group_18-24          50000 non-null  bool
 19  age_group_25-34          50000 non-null  bool
 20  age_group_35-44          50000 non-null  bool
 21  age_group_45-54          50000 non-null  bool
 22  age_group_55+            50000 non-null  bool
 23  traveller_type_Business  50000 non-null  bool
 24  traveller_type_Couple    50000 non-null  bool
 25  traveller_type_Family    50000 non-null  bool
 26  traveller_type_Solo      50000 non-null  bool
dtypes: bool(9), float64(13), int64(2), object(3)
memory usage: 7.3+ MB
None
```

10. **Preview of Cleaned Data:** The first five rows of the cleaned dataset are displayed .

	user_id	score_overall	score_cleanliness	score_comfort	score_facilities	score_location	score_staff	score_value_for_money	hotel_name	city	...	user_gender_encoded	age_group_18-24	age_group_25-34	age_group_35-44	age_group_45-54	age_group_55+	traveller_type_Business	traveller_type_Couple	traveller_type_Family	traveller_type_Solo
0	1600	8.7	8.6	8.7	8.5	9.0	8.8	8.7	The Azure Tower	New York	...	0	False	True	False	False	False	False	False	False	True
1	432	9.1	10.0	9.1	9.0	8.6	9.4	8.6	Kyo-to Grand	Tokyo	...	0	False	False	True	False	False	False	True	False	False
2	186	8.8	9.7	8.8	8.3	8.7	8.1	8.6	Nile Grandeur	Cairo	...	0	False	False	False	False	True	False	True	False	False
3	1403	8.9	9.0	8.8	8.5	9.6	9.1	8.3	Gaudi's Retreat	Barcelona	...	0	False	False	True	False	False	True	False	False	False
4	1723	9.1	8.9	9.5	9.3	8.3	9.4	8.9	Kremlin Suites	Moscow	...	1	False	False	False	True	False	False	False	True	False

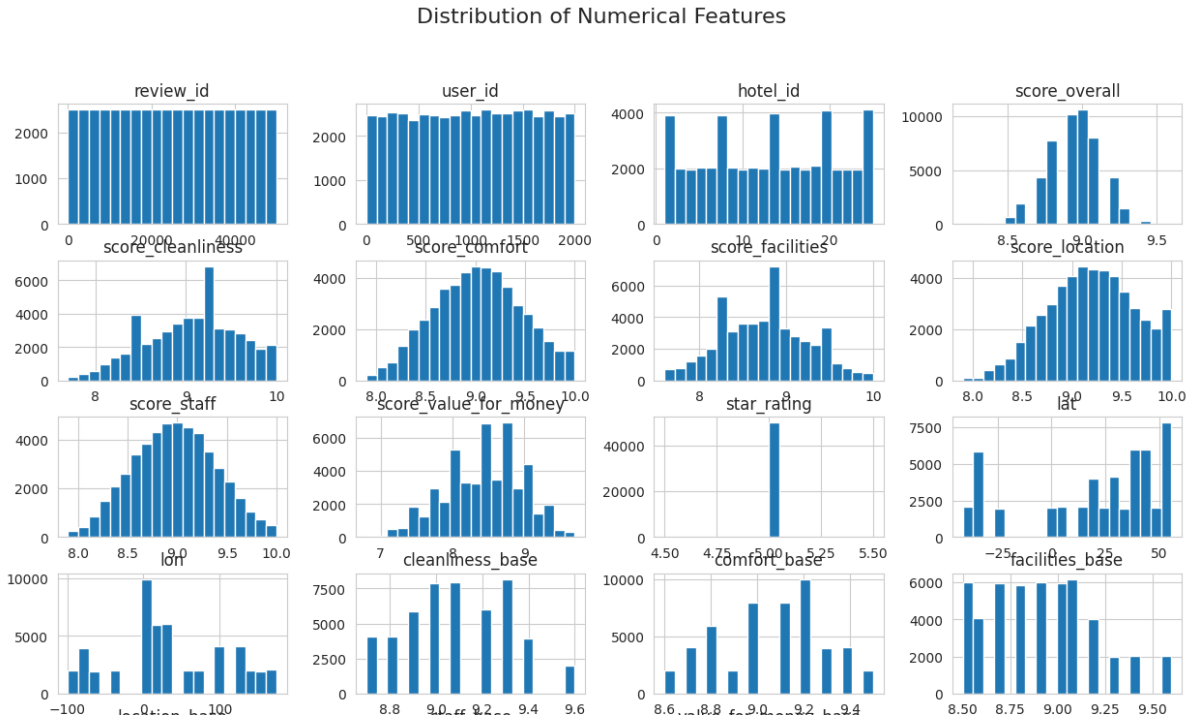
[5 rows x 27 columns]

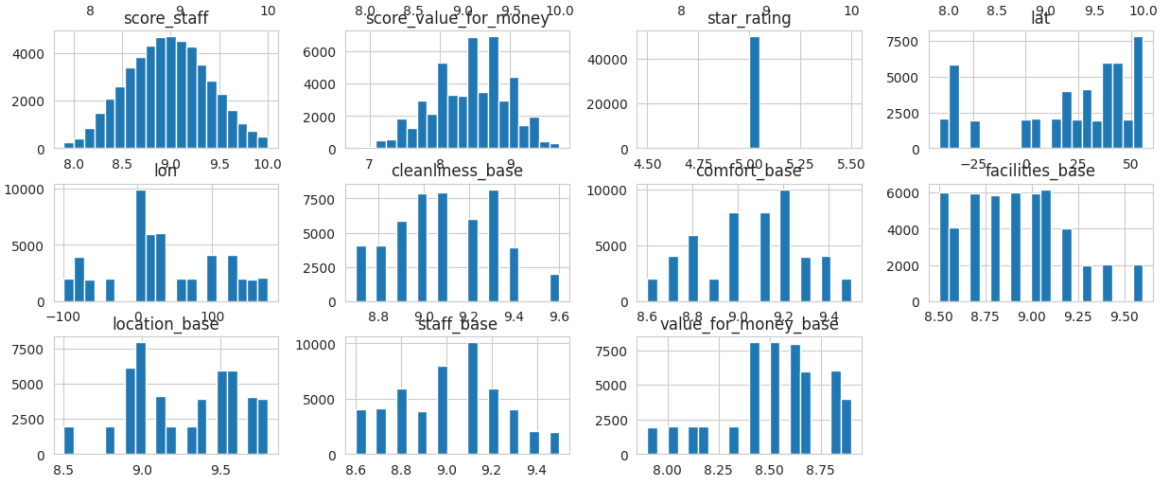
11. **Target Column Preparation:** Created a new empty column to later hold regional mapping:

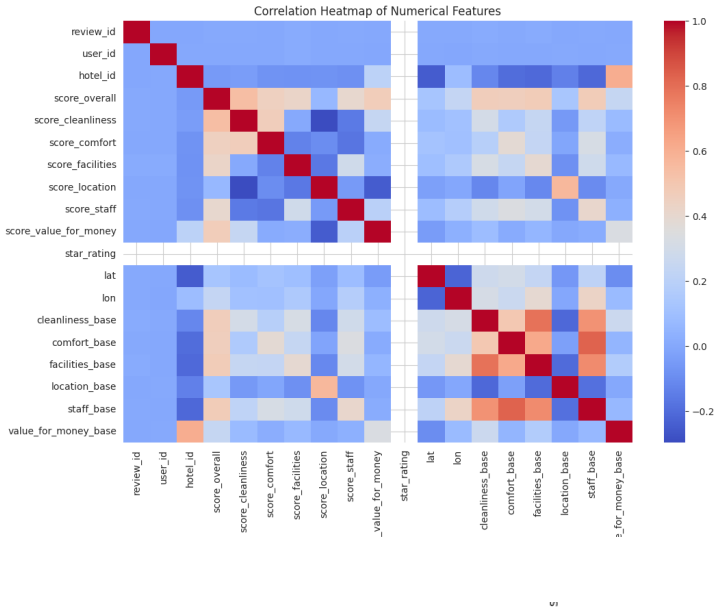
```
merged_df['country_group'] = ''
```

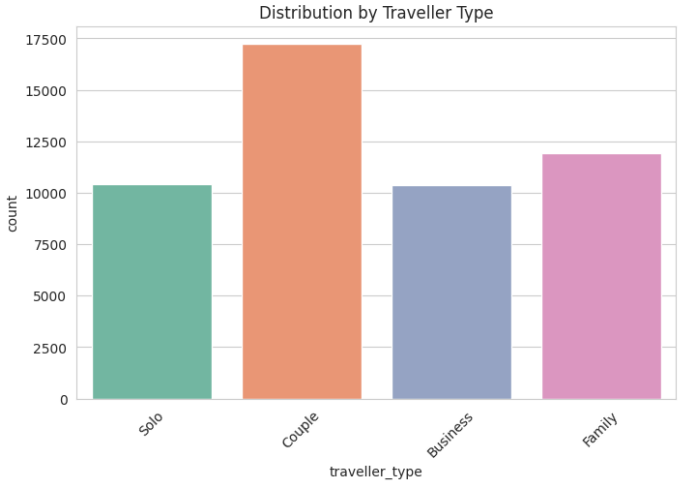
3.3 Exploratory Data Analysis and Feature Distributions

The exploratory data analysis reveals several key patterns in the hotel review dataset. The distribution of numerical features (Image 1 and Image 4) shows that most score variables (cleanliness, comfort, facilities, location, staff, and value for money) follow approximately normal distributions centered around 8.5-9.0 on a 10-point scale, indicating generally positive guest experiences across all evaluated aspects. The overall rating (score-overall) exhibits a similar pattern with a mean around 8.5-9.0, suggesting consistency between individual scores and overall satisfaction. Notably, the star-rating variable shows a strong concentration at the 5.0 level, indicating that the dataset primarily contains reviews from higher-tier hotels. The correlation heatmap (Image 2) demonstrates strong positive correlations among all score variables, with particularly high correlations between score-overall and individual service aspects (cleanliness, comfort, facilities, staff, and value for money), ranging from approximately 0.6 to 0.8. This suggests that these features are highly predictive of overall guest satisfaction. Interestingly, the base score variables (cleanliness-base, comfort-base, etc.) also show moderate to strong correlations with their corresponding review scores, indicating consistency in hotel quality over time. The distribution by traveller type (Image 3) reveals a fairly balanced dataset with slight variation: couples represent the largest segment (17,500 reviews), followed by family travelers (12,000), solo travelers (10,500), and business travelers (10,000), providing sufficient data across all travel categories for robust analysis.




score_staff
score_value_for_money
star_rating
lat
lon
cleanliness_base
comfort_base
facilities_base
location_base
staff_base
value_for_money_base


Correlation Heatmap of Numerical Features
review_id
user_id
hotel_id
score_overall
score_cleanliness
score_comfort
score_facilities
score_location
score_staff
score_value_for_money
star_rating
lat
lon
cleanliness_base
comfort_base
facilities_base
location_base
staff_base
value_for_money_base


Distribution by Traveller Type
count
Solo
Couple
Business
Family
traveller_type

3.4 Feature Engineering

3.4.1 Target Variable Creation

Mapped individual countries to 11 regional groups: North_America, Western_Europe, Eastern_Europe, East_Asia, Southeast_Asia, Middle_East, Africa, Oceania, South_America, South_Asia, and North_America_Mexico. The raw `country_x` column was dropped after encoding.

3.4.2 Feature Categories

1. Review Score Features (7 features):

- score_overall
- score_cleanliness
- score_comfort
- score_facilities
- score_location
- score_staff
- score_value_for_money

Rationale: These scores reflect hotel quality perceptions that may vary by region due to cultural standards and expectations.

2. User Demographics (11 features):

- user_gender_encoded (label encoded)
- age_group one-hot encoded (18–24, 25–34, 35–44, 45–54, 55+)
- traveller_type one-hot encoded (Business, Couple, Family, Solo)

*V*arious people of different demographics might favor certain areas, or they will not grade hotels in the same way, depending on cultural backgrounds.

3. Delta Features (6 features) – Advanced Model Only:

$$\begin{aligned}
\text{delta_cleanliness} &= \text{score_cleanliness} - \text{cleanliness_base} \\
\text{delta_comfort} &= \text{score_comfort} - \text{comfort_base} \\
\text{delta_facilities} &= \text{score_facilities} - \text{facilities_base} \\
\text{delta_location} &= \text{score_location} - \text{location_base} \\
\text{delta_staff} &= \text{score_staff} - \text{staff_base} \\
\text{delta_value_for_money} &= \text{score_value_for_money} - \text{value_for_money_base}
\end{aligned}$$

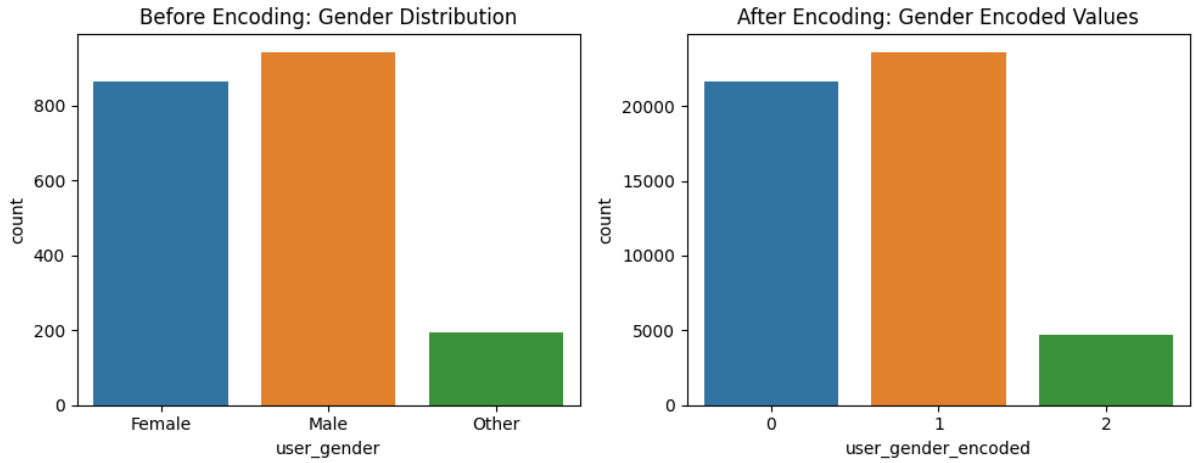
*M*easures individual review deviation against hotel baseline averages, indicating the sentiment of the reviewers as compared to the standards of the hotels. This assists in finding out the regional rating patterns.

Total Features:

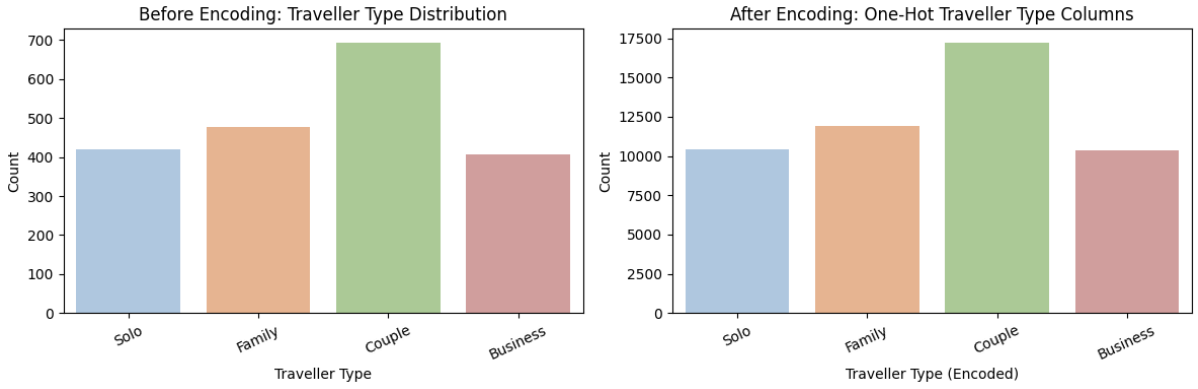
- Baseline models: 18 features (scores + demographics)
- Advanced models: 24 features (scores + demographics + deltas)

3.4.3 Visual Effect of Preprocessing

To visualize how preprocessing affected the dataset, we included two examples that show different transformation types applied during data preparation.



The first figure compares the gender distribution before and after encoding. The "Before Encoding" plot shows the original text categories, Female, Male, and Other, while the "After Encoding" plot shows their corresponding numeric codes (0, 1, and 2). The proportions remained almost identical, which confirms that encoding successfully transformed the categorical variable into a machine-readable format without changing its original balance or meaning.



The second figure compares the traveller type distribution before and after encoding. The left plot shows the original categories, Solo, Family, Couple, and Business, where Couples are the most common and Business travellers are the least. The right plot shows the same distribution after one-hot encoding, where each traveller type becomes a numeric column. Although the total counts increased because the merged dataset includes many more records, the proportions remained consistent, confirming that encoding preserved the original data balance while converting the values into a machine-readable format.

Together, these two visualizations confirm that the preprocessing and feature-engineering steps preserved the natural structure of the data while making it suitable for model training and interpretation.

3.5 Model Development

3.5.1 Data Splitting Strategy

- Train-Test Split: 80–20 stratified by target class
- Validation Split: 20% of training data for hyperparameter tuning
- Stratification ensures balanced class representation

3.5.2 Models Implemented

1. Logistic Regression (Multinomial)

- Solver: lbfgs
- Max iterations: 1000
- Multi-class: softmax formulation
- No regularization tuning (baseline model)

2. Random Forest Classifier

- n_estimators: 600 trees
- max_depth: None (full growth)
- min_samples_split: 4
- min_samples_leaf: 2
- max_features: sqrt
- class_weight: balanced (handles class imbalance)

3. Neural Network (Feedforward)

- Architecture: Input → Dense(32, ReLU) → Dense(16, ReLU) → Dense(num_classes, Softmax)
- Optimizer: Adam
- Loss: Sparse categorical crossentropy
- Early stopping: patience=3, monitor validation accuracy
- Max epochs: 50

3.5.3 Evaluation Metrics

Accuracy, Precision, Recall, F1-Score, Confusion matrices, and Classification reports per class.

3.6 Model Explainability

SHAP (SHapley Additive exPlanations):

- TreeExplainer for Random Forest
- Global feature importance via summary plots
- Local instance explanations via force plots

LIME (Local Interpretable Model-agnostic Explanations):

- LimeTabularExplainer with discretized continuous features
- Instance-level feature contribution analysis
- Top 8 features displayed per prediction

4 Data-Engineering Questions

4.1 Which city is best for each traveler type

To each type of traveler, we filtered the dataset such that type = 1 following one-hot encoding. After this, we obtained the average overall review score per city in order to determine which city suits each type of traveller. Each type of traveler was chosen in the highest-scoring city to find the best destinations in each type of traveler. This assists in comparing preferences of such groups as Business, Family, Solo, and Couple travelers. We also applied a barplot, as it is a good way to compare the categorical groups (types of travelers) with a numerical metric (average score). Barplots attached below are simple visual ranking and comparison of scores

in categories. According to the output, Dubai is the best destination in terms of Business and Family travelers, and Amsterdam is the best destination in terms of Solo and Couple travelers, indicating varying preferences by the type of traveler.

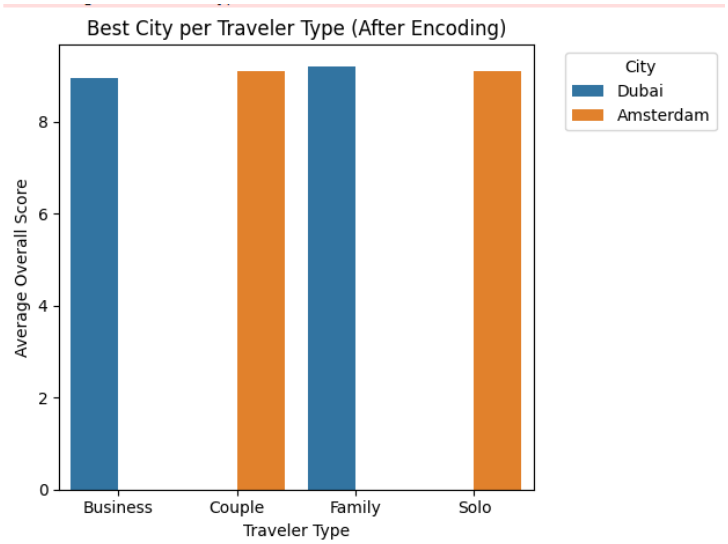


Figure 1: Relation between Average Overall Score and Traveler Type

4.2 What are the top 3 countries with the best value-for-money score per traveler's age group?

Prepared a blank DataFrame so I could store the leading countries of each age group in terms of average value-for-money score, and I could store the results in a format that I can analyze. Each age group filtered the dataset to only get travelers in that group and calculated the average value-for-money score in each country. Choose the top 3 countries with the highest average scores by each age group in order to show the best destinations by each segment of the traveler. Ranked and re-ranked the results in order to preserve the clear order and guarantee the consistency of alignment among the age-country combinations. A barplot was used since it is easy to visually compare nominal groups (age groups) with numeric values (value-for-money) and to understand the visual trends. The results of the output indicate that the leading destinations differ with age group - the same countries emerge as leading (China and the Netherlands) all the time, whereas other countries are changing with age groups.

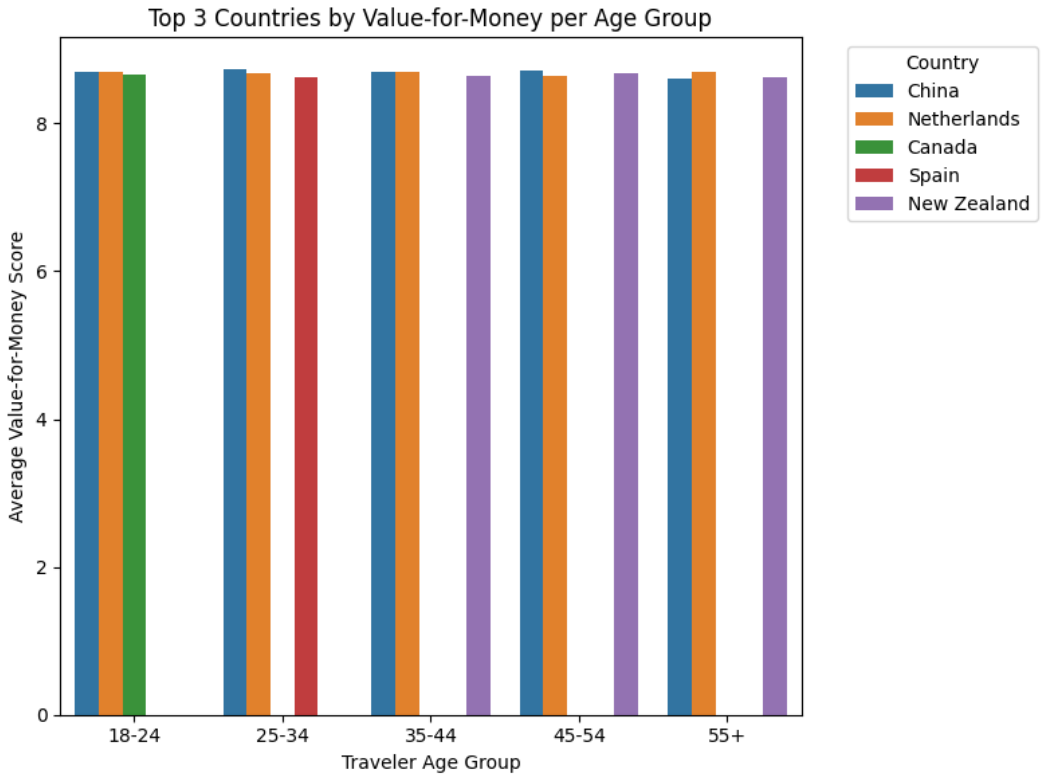


Figure 2: Average Value-for-Money Score VS Traveler Age Group

5 Results

5.0.1 Scenario 1: shallow FFNN Model

All score-based (hotel info + user review) and user features have been selected to be features of training, in addition to the Neural Network model details described before in section 3 used here. The results shows validation accuracy 95% and test accuracy 98% .

✅ Final Validation Accuracy: 0.9501
✅ Overall Test Accuracy: 0.9801

Detailed classification report:

	precision	recall	f1-score	support
Africa	0.98	0.86	0.92	1226
East_Asia	1.00	1.00	1.00	1216
Eastern_Europe	1.00	1.00	1.00	394
Middle_East	1.00	1.00	1.00	797
North_America	1.00	1.00	1.00	792
North_America_Mexico	1.00	1.00	1.00	401
Oceania	1.00	1.00	1.00	803
South_America	1.00	1.00	1.00	784
South_Asia	1.00	0.95	0.98	398
Southeast_Asia	1.00	1.00	1.00	814
Western_Europe	0.93	1.00	0.96	2375
accuracy			0.98	10000
macro avg	0.99	0.98	0.99	10000
weighted avg	0.98	0.98	0.98	10000

Figure 3: Classification Report for shallow FFNN Model

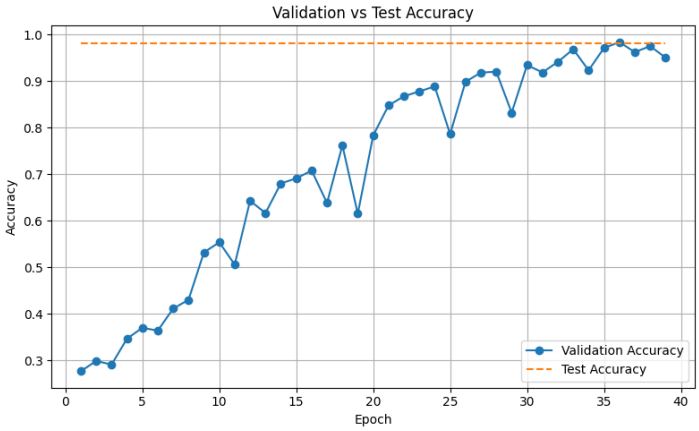


Figure 4: Validation vs test accuracy graph for shallow FFNN Model

While the model achieved a high test accuracy further analysis suggested potential data leakage.Features such as the hotel's baseline quality metrics (e.g., cleanliness-base, staff-base) are unique to each property and therefore indirectly reveal the hotel's location.

5.0.2 Scenario 2: Multinomial Logistic Regression Model

In the scenario, a Logistic Regression model will be trained on the user and review-based features, without the help of hotel-specific features to get a more realistic and generalizable predictive performance measure.Achieves an accuracy 34%. As seen in the matrix there 're a huge conflit and miss prediction between classes.

✅ Accuracy: 0.3447
✅ Precision: 0.2801
✅ Recall: 0.3447
✅ F1 Score: 0.2753

Detailed classification report:

	precision	recall	f1-score	support
Africa	0.34	0.49	0.40	1226
East_Asia	0.33	0.46	0.38	1216
Eastern_Europe	0.18	0.03	0.05	394
Middle_East	0.26	0.06	0.10	797
North_America	0.23	0.09	0.13	792
North_America_Mexico	0.29	0.05	0.09	401
Oceania	0.00	0.00	0.00	803
South_America	0.26	0.17	0.21	784
South_Asia	0.00	0.00	0.00	398
Southeast_Asia	0.40	0.18	0.25	814
Western_Europe	0.37	0.78	0.50	2375
accuracy			0.34	10000
macro avg	0.24	0.21	0.19	10000
weighted avg	0.28	0.34	0.28	10000

Figure 5: Classification Report for Logistic Regression Model

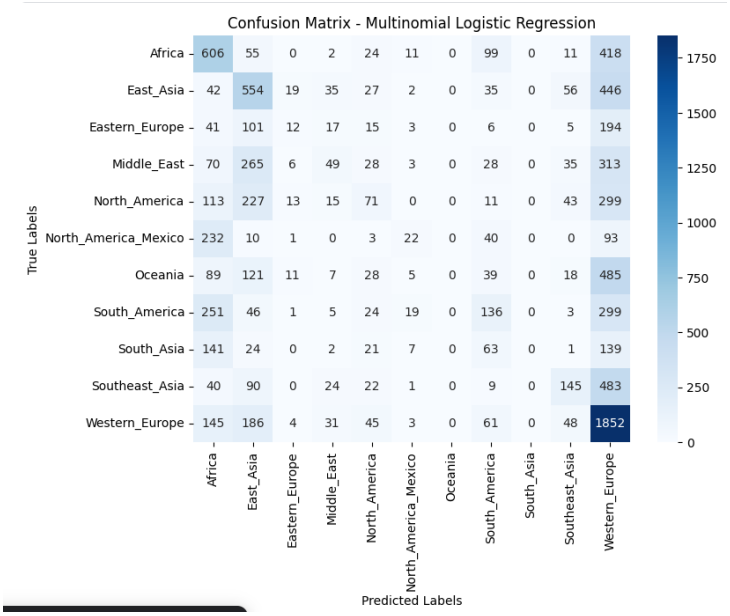


Figure 6: Confusion Matrix for Logistic Regression Model

5.0.3 Scenario 3: Shallow FFNN but without the base scores

Trained a small feed-forward neural network (32→16→softmax) on only review score features (overall, cleanliness, comfort, facilities, location, staff, value-for-money) plus basic demographics (gender, age one-hots, traveller type one-hots) to predict the country group. We used a stratified split, early stopping, and standard cross-entropy—no hotel/base features or engineered interactions. Validation accuracy is 28.5% and test accuracy is 29.7% with many classes having precision/recall = 0 (warnings confirm some classes were never predicted). The model heavily favors the dominant class (Western Europe) and can't separate the others—clear sign that the current inputs don't carry enough region-specific signal. we need more informative inputs related to geography/hotel context

✅ Final Validation Accuracy: 0.2850
✅ Overall Test Accuracy: 0.2978

Detailed classification report:

	precision	recall	f1-score	support
Africa	0.23	0.06	0.09	1226
East_Asia	0.40	0.39	0.40	1216
Eastern_Europe	1.00	0.00	0.01	394
Middle_East	0.31	0.04	0.07	797
North_America	0.10	0.00	0.00	792
North_America_Mexico	0.00	0.00	0.00	401
Oceania	0.00	0.00	0.00	803
South_America	0.25	0.12	0.16	784
South_Asia	0.00	0.00	0.00	398
Southeast_Asia	0.34	0.18	0.23	814
Western_Europe	0.28	0.91	0.43	2375
accuracy			0.30	10000
macro avg	0.26	0.15	0.13	10000
weighted avg	0.26	0.30	0.20	10000

Figure 7: Classification Report for shallow FFNN Model without the base scores

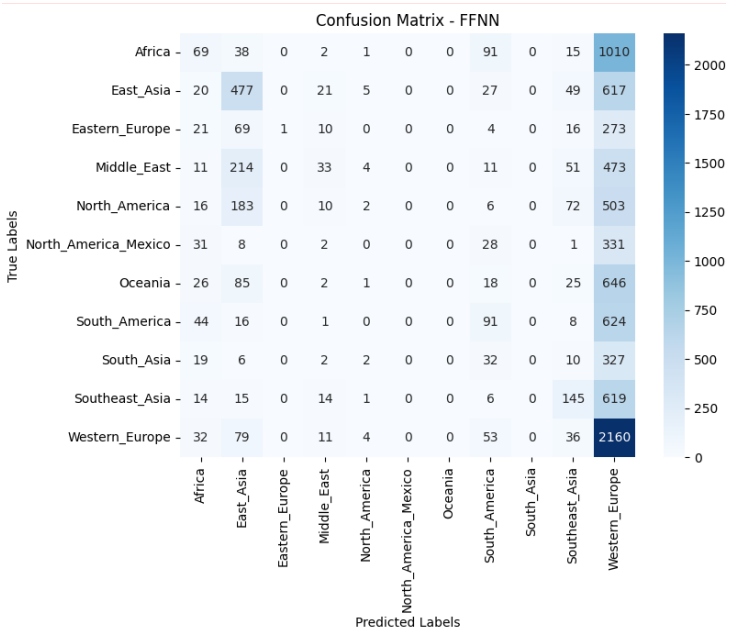


Figure 8: Confusion Matrix for shallow FFNN Model without the base scores

5.0.4 Scenario 4: Shallow FFNN but with deltas

The goal for using deltas was to inject hotel-relative signal (how a guest's score deviates from the hotel's baseline) without feeding the raw base columns directly. validation accuracy is 39% and test accuracy is 40.7%,

We picked random 2 columns from the dataset to test model predict both incorrectly which indicate that this model can't be the one to be used in addition to the matrix which help us to visualize the miss prediction between classes. For the first raw the prediction should be Africa while for the second one the prediction should be North-America-Mexico.

Detailed classification report:

	precision	recall	f1-score	support
Africa	0.39	0.26	0.31	1226
East_Asia	0.32	0.71	0.44	1216
Eastern_Europe	0.00	0.00	0.00	394
Middle_East	0.00	0.00	0.00	797
North_America	0.00	0.00	0.00	792
North_America_Mexico	0.00	0.00	0.00	401
Oceania	0.00	0.00	0.00	803
South_America	0.47	0.82	0.60	784
South_Asia	0.00	0.00	0.00	398
Southeast_Asia	0.91	0.16	0.28	814
Western_Europe	0.42	0.90	0.58	2375
accuracy			0.41	10000
macro avg	0.23	0.26	0.20	10000
weighted avg	0.30	0.41	0.30	10000

Figure 9: Classification Report for Shallow FFNN but with deltas

Figure 10: Confusion Matrix for Shallow FFNN but with deltas

```
    "facilities_base": 9.1,
    "value_for_money_base": 8.4,
    "user_gender": "Female",
    "age": 60,
    "traveller_type": "Couple"
}

print(infer_country_group(example_input_2))

example_input_6 = {
    "score_overall": 8.7,
    "score_cleanliness": 9.0,
    "score_comfort": 9.2,
    "score_facilities": 8.8,
    "score_location": 8.6,
    "score_staff": 8.5,
    "score_value_for_money": 8.2,
    "comfort_base": 8.9,
    "facilities_base": 8.5,
    "value_for_money_base": 8.5,
    "user_gender": "Female",
    "age": 39,  # age_group = "35-44"
    "traveller_type": "Family"
}
# 🏨 country_x = "Mexico"

print(infer_country_group(example_input_6))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
🌍 Predicted Country Group: **North_America** (confidence: 100.00%)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
🌍 Predicted Country Group: **Oceania** (confidence: 54.10%)
```

+ Code + Markdown

Figure 11: Test samples from the dataset to verify the FFNNmodel

5.0.5 Scenario 5: Random Forest

In this experiment, a Random Forest Classifier was trained using only the score-based features (hotel review scores) and user demographics (age, gender, traveler type) to predict the hotel's region (country group).This model was chosen because Random Forests handle nonlinear relationships and mixed data types well, and they provide feature importance insights, making them a strong baseline for structured data. The aim was to see if traditional tabular learning could outperform neural networks when only using basic features without modality-style enhancement.Validation accuracy is 39% and test accuracy is 37%. We have chosen 5 other random samples from the dataset and showed 4 predict correctly even with low confidence rate prove very low percentage of data leakage.

	precision	recall	f1-score	support
Africa	0.40	0.41	0.41	1226
East_Asia	0.45	0.44	0.45	1216
Eastern_Europe	0.17	0.17	0.17	394
Middle_East	0.38	0.28	0.32	797
North_America	0.38	0.34	0.36	792
North_America_Mexico	0.23	0.26	0.24	401
Oceania	0.32	0.21	0.25	803
South_America	0.25	0.30	0.27	784
South_Asia	0.12	0.11	0.11	398
Southeast_Asia	0.33	0.32	0.32	814
Western_Europe	0.48	0.56	0.51	2375
accuracy			0.37	10000
macro avg	0.32	0.31	0.31	10000
weighted avg	0.37	0.37	0.37	10000

Figure 12: Classification Report for Random Forest Model

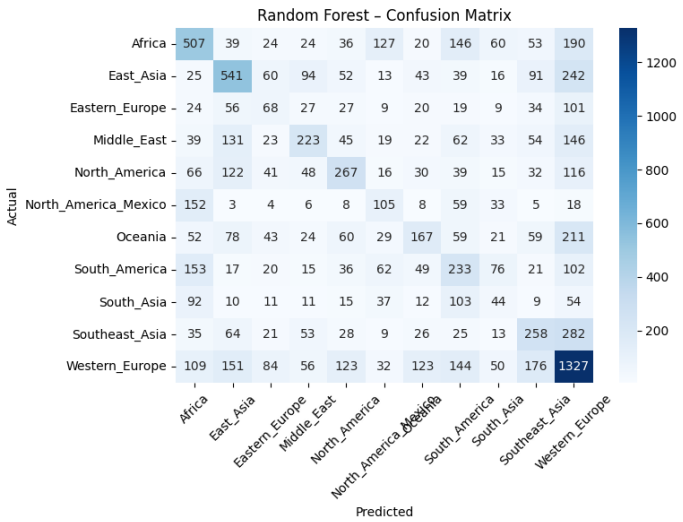


Figure 13: Confusion Matrix for Random Forest Model

```
Top feature importances:
 score_value_for_money    0.133941
score_facilities          0.132237
score_location            0.131864
score_cleanliness         0.122938
score_comfort             0.121522
score_staff               0.120698
score_overall             0.068567
user_gender_encoded       0.046192
age_group_25-34           0.025889
age_group_35-44           0.025011
dtype: float64
```

Figure 14: Rate features effect the Random Forest model the most

```
Predicted Country Group: **Africa** (confidence: 28.69%)
Predicted Country Group: **East_Asia** (confidence: 31.42%)
Predicted Country Group: **Southeast_Asia** (confidence: 41.42%)
Predicted Country Group: **Africa** (confidence: 16.65%)
Predicted Country Group: **North_America_Mexico** (confidence: 58.60%)
```

Figure 15: Test the model on real dataset samples

5.1 Model Explainability Insights

5.1.1 SHAP

Use TreeExplainer for specific classt on real dataset samples in RandomForest, and keep background very small for speed use only 100 sample which gives an overview about the model then Compute SHAP values for only 100 test samples.Also we have to disable strict additivity check that's because decreases chance of rounding errors

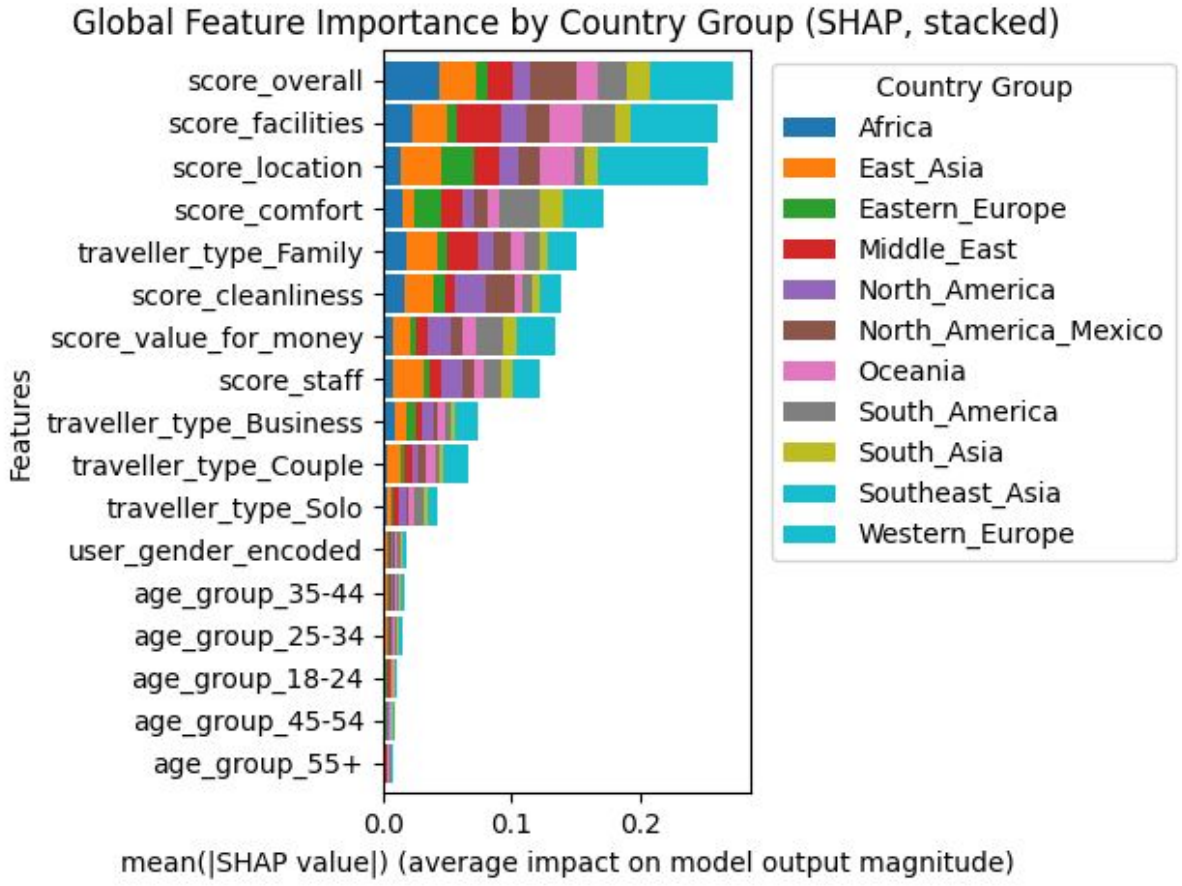


Figure 16: Analyze BlackBox using SHAP

5.1.2 LIME

Lime used for multi-Class for Random Forest by create LIME explainer using NumPy array,feature names for display,country group labels,split continuous features ,and into bins. Then create function that returns class probabilities for RandomForestClassifier. In addition to that specify to show a top 3 predicted countries.

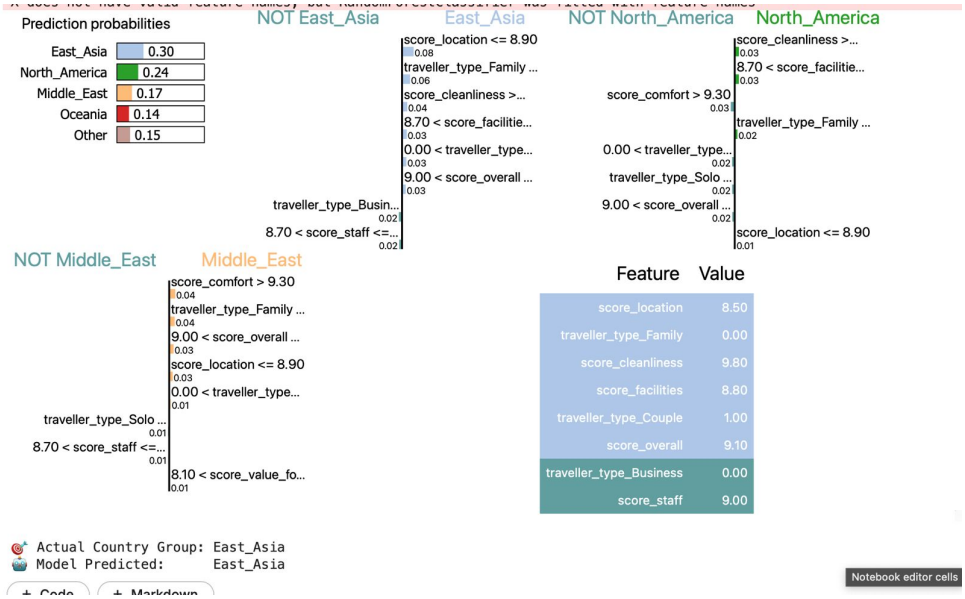


Figure 17: Analyze BlackBox using LIME

6 Inference Function

To make our trained model usable on new and unseen data, we created an inference function called infercountrygroup(). This function takes raw inputs, such as the hotel scores, hotel base scores, user gender, age or age group, and traveller type, and automatically applies the same preprocessing steps that were used before model training. It checks if all required information is available, validates the values, and handles any missing inputs by assigning them to the correct format. Then, it converts categorical data such as gender and traveller type into numeric values, creates one hot encoded columns for each age group and traveller type, and computes the delta features that show the difference between the user's rating and the hotel's usual base score. After preparing the input data, the function passes it to the trained Random Forest model to generate a prediction. The output shows the most likely country group (or region) along with a confidence percentage, allowing the model to give meaningful, human readable results. This makes the function practical for real use, since it can take any new review and directly tell which region the hotel most likely belongs to, without any manual work.

We then tested it using several random real examples that covered different traveller types, ages, and genders. The function was able to predict the country group for each input and provided the confidence of each prediction. In most of the cases, the model predicted the correct region: for example, a 60 year old female traveller who travelled as a couple was predicted as **Africa** with 28.69% confidence; a 29 year old male couple traveller was predicted as **Southeast Asia** with 41.42% confidence; a 27 year old female solo traveller was predicted as **Africa** with 16.65% confidence; and a 39 year old female family traveller was predicted as **North America (Mexico)** with 58.60% confidence.

7 Conclusion

To sum up based on the screenshots and the details we have provided above, Random Forest model with Validation accuracy is 39% and test accuracy is 37% is best one to use for predicting the country group based on hotel's reviews with using Model Explainability techniques which verifying and explain how the model work.